Advanced AI Job Application Bot

Intelligent Job Matching & Automated Application System

Version 2.0 | December 2025

Developer Information

Vignesh Yadala

vignesh.yadala@gmail.com

linkedin.com/in/vignesh-yadala

github.com/Vigneshyadala

+91 9032478898

Python | AI | Automation

📊 Executive Summary

🎯 Project Vision

The Advanced AI Job Application Bot represents a cutting-edge solution in career automation, leveraging artificial intelligence to revolutionize the job search process. This sophisticated system combines web scraping, natural language processing, and intelligent decision-making to streamline job applications across multiple platforms.

Key Achievements

15+	100%	2,500+
Companies Supported	AI-Powered	Lines of Code

Problem Statement

Job seekers face significant challenges in today's competitive market:

- Time-consuming manual job searches across multiple platforms
- Difficulty identifying relevant opportunities matching skillsets
- Repetitive application processes across different portals
- Lack of intelligent filtering and prioritization
- Limited tracking and analytics capabilities

Solution Overview

Automated Job Discovery

Scrapes opportunities from Naukri.com and 14+ major tech companies with intelligent filtering

AI-Powered Analysis

Uses Google Gemini AI to evaluate job-resume compatibility with 0-100% scoring

Intelligent Application

Automatically applies to high-match positions with configurable thresholds

Professional Dashboard

Flask-based web interface with real-time analytics and visualizations

🏗️ System Architecture

Core Components

1

Bot Engine (enhanced_main.py)

Lines of Code: 689 lines

Orchestrates job scraping, AI analysis, and application automation. Handles Playwright browser control, resume parsing, and multi-company job discovery.

2

Web Backend (app.py)

Lines of Code: 487 lines

Flask REST API managing data flow, configuration, and bot control. Provides 10+ endpoints for dashboard interaction and real-time status updates.

3

Dashboard (index.html)

Lines of Code: 1,245 lines

Modern web interface with analytics, company selection, and control panel. Built with vanilla JavaScript and Chart.js for visualizations.

4

Analytics Dashboard (dashboard.py)

Lines of Code: 143 lines

CLI-based analytics viewer for quick statistics and insights without launching web interface.

Data Flow Architecture

```
# User uploads resume via web interface Resume (PDF) → PyPDF2 → Text Extraction # Bot scrapes jobs from multiple sources Job Portals → Playwright → Job Data (JSON/CSV) # AI analyzes each job Job + Resume → Google Gemini → Match Score + Reasoning # Results stored and displayed Analysis Results → CSV/JSON → Dashboard Visualization
```

Supported Platforms

🌐 Job Portal

Naukri.com: India's #1 job portal with 70M+ registered users and comprehensive listings across all industries

IN Indian IT Giants (7)

- TCS - Tata Consultancy Services
- Infosys
- Wipro
- HCLTech
- Tech Mahindra
- Accenture India
- Cognizant

🌍 FAANG + Big Tech (7)

- Google
- Microsoft
- Amazon
- Apple
- Meta (Facebook)
- Oracle
- IBM

✨ Key Features & Capabilities

🤖 AI-Powered Job Matching

- Google Gemini 1.5 Flash integration
- 0-100% match scoring system
- Detailed reasoning for recommendations
- Skills, experience & role analysis

🔍 Smart Job Discovery

- Multi-role simultaneous searching
- Cross-location aggregation
- Experience-based filtering
- Company-specific scraping

⚡ Automated Application

- One-click multi-job application
- Playwright browser automation
- Auto-login & session management
- Success verification tracking

📊 Professional Dashboard

- Real-time statistics display
- Interactive Chart.js visualizations
- Application history tracking
- Company-wise analytics

Resume Management

- Drag-and-drop PDF upload
- Automatic text extraction
- Resume preview & validation
- 16MB file size support

Configuration System

- Web-based settings editor
- Customizable match thresholds
- Daily application limits
- Multi-company selection UI

Advanced Analytics

- Match score distribution charts
- Company-wise statistics
- Location-based insights
- Timeline visualization

Safety Features

- Configurable application limits
- Duplicate job prevention
- Manual approval mode
- Comprehensive error handling

User Interface Highlights

🎨 Enhanced Company Selection UI

- **Categorized Display:** Jobs Portal, Indian IT Giants, FAANG+
- **Visual Selection:** Gradient-styled active states with animations
- **Bulk Actions:** "Select All" per category, "Clear All" button
- **Selection Counter:** Real-time count of selected companies

- **Company Icons:** Visual emoji identifiers for quick recognition

🛠 Technology Stack

Core Technologies

Python 3.8+ | Playwright | Flask 2.3 | Google Gemini AI | PyPDF2

Pandas | HTML5/CSS3 | JavaScript ES6 | Chart.js | REST API

Detailed Technology Breakdown

Technology	Purpose	Version
Playwright	Browser automation & web scraping	1.40+
Flask	Web server & REST API backend	2.3.0
Google Gemini AI	AI-powered job-resume matching	1.5 Flash
PyPDF2	PDF resume text extraction	3.0.1
Pandas	Data analysis & CSV handling	2.0.0
Chart.js	Dashboard data visualizations	4.4.0
Python-dotenv	Environment variable management	1.0.0

Project Structure

```
ai-job-application-bot/ | ├── enhanced_main.py # Main bot engine (689 lines) ├──
app.py # Flask web server (487 lines) ├── dashboard.py # CLI analytics (143 lines) |
├── templates/ | └── index.html # Web dashboard (1,245 lines) | ├── data/ | ├──
resume.pdf # User resume file | ├── job_results_*.csv # Application results | └──
applied_jobs.json # Tracking data | ├── logs/ | └── bot_*.log # Daily activity logs |
```

```
├── .env # Configuration file ├── requirements.txt # Python dependencies └── 
README.md # Documentation
```

💻 Technical Implementation

1. Resume Processing Module

```
import PyPDF2 def extract_resume_text(pdf_path): """Extract text from PDF resume"""
with open(pdf_path, 'rb') as file: pdf_reader = PyPDF2.PdfReader(file) text = "" for
page in pdf_reader.pages: text += page.extract_text() return text
```

2. AI Job Matching Engine

```
import google.generativeai as genai genai.configure(api_key=GOOGLE_API_KEY) model =
genai.GenerativeModel('gemini-1.5-flash') def analyze_job_with_ai(job_data,
resume_text): prompt = f""" Analyze job posting and resume match: JOB:
{job_data['title']} at {job_data['company']} LOCATION: {job_data['location']}
EXPERIENCE: {job_data['experience']} CANDIDATE RESUME: {resume_text[:2000]} Provide:
- Match score (0-100%) - Detailed reasoning - Apply recommendation (yes/no) """
response = model.generate_content(prompt) return parse_ai_response(response.text)
```

3. Web Scraping with Playwright

```
from playwright.sync_api import sync_playwright def scrape_naukri_jobs(page, role,
location): url = f"https://www.naukri.com/{role}-jobs-in-{location}" page.goto(url)
job_cards = page.query_selector_all(".job-card") jobs = [] for card in job_cards: job
= { 'title': card.query_selector('.title').text(), 'company':
card.query_selector('.company').text(), 'location':
card.query_selector('.location').text() } jobs.append(job) return jobs
```

4. Flask API Endpoints

`GET` **/api/stats** - Retrieve application statistics

POST **/api/resume/upload** - Upload resume PDF

POST **/api/start-bot** - Start bot with configuration

GET **/api/bot-status** - Check bot running status

🚀 Installation & Setup Guide

Prerequisites

- Python 3.8 or higher
- pip (Python package manager)
- Google Gemini API key (free tier available)
- Chrome/Chromium browser (auto-installed)
- 4GB RAM minimum (8GB recommended)

Step-by-Step Installation

1

Clone Repository

```
git clone https://github.com/Vigneshyadala/ai-job-bot.git cd ai-job-bot
```

2

Install Python Dependencies

```
pip install -r requirements.txt playwright install chrome
```

3

Configure Environment Variables

Create `.env` file in project root:

```
GOOGLE_API_KEY=your_gemini_api_key_here
NAUKRI_EMAIL=your_email@example.com NAUKRI_PASSWORD=your_password
DESIRED_ROLES=Python Developer,Software Engineer
DESIRED_LOCATIONS=Bangalore,Mumbai,Chennai MIN_EXPERIENCE=0
MAX_EXPERIENCE=5 MIN_MATCH_SCORE=75 MAX_APPLICATIONS_PER_DAY=20
```

4

Prepare Resume

```
mkdir data cp your_resume.pdf data/resume.pdf
```

5

Launch Application

```
python app.py
```

Access dashboard at: **http://localhost:5000**

📖 Usage Guide

Complete Workflow

1

Upload Resume

- Navigate to "Resume" tab
- Click upload area or drag PDF file
- Confirm successful upload (✅ notification)
- Resume automatically parsed and validated

2

Configure Settings

- Go to "Settings" tab
- Enter Google Gemini API key
- Set minimum match score (recommended: 75%)
- Set max applications per day (recommended: 20)
- Click "Save Settings"

3

Select Target Companies

- Open "Control Panel" tab
- Choose companies individually or use "Select All"
- Naukri.com is selected by default
- Options: Indian IT Giants, FAANG+, or custom selection

4

Configure Job Preferences

- Enter desired roles (comma-separated)
- Enter target locations
- Set experience range (min-max years)

5

Start Bot

- Review selections (check counter)
- Click "Start Job Application Bot"
- Browser opens automatically
- Monitor progress in terminal/logs

6

View Results

- Switch to "Overview" tab
- View real-time statistics
- Analyze charts and trends
- Export data from CSV files

📊 Data Analytics & Reporting

Metrics Tracked

Metric	Description
Total Applications	Count of all jobs analyzed by the bot
Successful Applications	Jobs where bot successfully applied
Average Match Score	Mean compatibility percentage across all jobs
Company Distribution	Jobs per company breakdown
Score Distribution	Jobs categorized by score ranges
Location Analysis	Geographic distribution of opportunities

Export Formats

📄 CSV Export

Filename: job_results_YYYYMMDD_HHMMSS.csv

Columns: title, company, location, experience, url, job_id, source, search_role, search_location, description, match_score, reasoning, should_apply, applied, timestamp

📊 JSON Export

Filename: job_results_YYYYMMDD_HHMMSS.json

Structure: Hierarchical format for programmatic access

Dashboard Visualizations

- **Bar Chart:** Applications by company (top 10)
- **Doughnut Chart:** Match score distribution (4 ranges)
- **Pie Chart:** Location-based distribution
- **Line Chart:** Application timeline
- **Statistics Cards:** Key performance indicators

🔐 Security & Privacy

Data Protection Measures

✅ Local Storage Only

All data (resume, credentials, results) stored locally on user machine. No cloud sync or external uploads.

✅ Encrypted API Communication

All API calls to Google Gemini use HTTPS encryption. API keys stored in environment variables.

✅ Credential Protection

Passwords and API keys stored in .env file (git-ignored). Never logged or transmitted insecurely.

Best Security Practices

1. **Never commit .env file** to version control
2. **Use strong API keys** and rotate regularly
3. **Keep dependencies updated** for security patches
4. **Review logs regularly** for suspicious activity
5. **Limit application rates** to avoid account flagging

Privacy Compliance

- No personal data transmitted to third parties
- Resume text sent only to Google Gemini API (encrypted)
- Job portal credentials used only for authentication
- All data deletable by user at any time

🔧 Troubleshooting Guide

Common Issues & Solutions

❌ Issue: Bot fails to start

Solution:

- Verify GOOGLE_API_KEY is set in .env
- Check internet connection
- Ensure Python 3.8+ is installed
- Run: `pip install -r requirements.txt`

❌ Issue: Resume upload fails

Solution:

- Ensure PDF is under 16MB
- Verify PDF contains searchable text (not scanned image)
- Check data/ folder permissions

❌ Issue: No jobs found

Solution:

- Try different role/location combinations
- Check spelling in job preferences
- Verify internet connectivity

- Try selecting more companies

❌ Issue: Charts not displaying

Solution:

- Clear browser cache (Ctrl+F5)
- Check browser console for errors (F12)
- Ensure Chart.js CDN is accessible
- Try different browser

📈 Performance Benchmarks

Speed Metrics

10-15
Jobs/Minute

2-3s
AI Analysis Time

<1s
PDF Processing

Resource Usage

Resource	Typical Usage	Peak Usage
Memory (RAM)	200-300 MB	500 MB
CPU	10-20%	40%
Disk Space	50-100 MB	500 MB (with logs)
Network	1-2 Mbps	5 Mbps

Scalability

- Can process 1000+ jobs per session
- Supports concurrent company scraping
- Efficient data caching mechanisms
- Handles large resume files (up to 16MB)

🚀 Future Enhancements

Planned Features (Roadmap)

🔗 LinkedIn Integration

Direct application on LinkedIn with Easy Apply automation

🌐 Indeed Support

Add Indeed.com scraping and application

📧 Email Notifications

Daily summary reports and application confirmations

📝 Cover Letter Generator

AI-generated custom cover letters per application

📱 Mobile App

React Native mobile version for iOS/Android

☁️ Cloud Deployment

Heroku/AWS deployment with 24/7 operation

🔄 Interview Scheduler

Automatic interview scheduling integration

📊 Advanced Analytics

ML-powered insights and predictions

📊 Project Statistics

2,564 Total Lines of Code	**4** Core Files	**10+** API Endpoints
15 Supported Platforms	**40+** Development Hours	**100%** AI-Powered

Skills Demonstrated

1. **Web Scraping:** Advanced DOM manipulation with Playwright
2. **AI Integration:** Google Gemini API implementation
3. **Full-Stack Development:** Flask backend + Modern frontend
4. **Data Analysis:** Pandas, CSV processing, statistics
5. **UI/UX Design:** Responsive, professional interface
6. **API Development:** RESTful architecture design
7. **Automation:** Browser automation, task scheduling
8. **File Handling:** PDF parsing, data storage

Development Timeline

Phase	Duration	Deliverable
Planning & Research	5 hours	Architecture design

Core Bot Development	15 hours	Scraping & AI integration
Web Dashboard	12 hours	Flask API + Frontend
Testing & Debugging	8 hours	Bug fixes & optimization
Total	**40 hours**	**v2.0 Release**

📜 License & Legal

Software License

MIT License

Free for personal and commercial use with attribution

Copyright © 2025 Vignesh Yadala

Terms of Use

1. Use responsibly and ethically
2. Respect website terms of service
3. Don't spam or abuse job portals
4. Provide accurate information in applications
5. Follow local employment laws
6. No warranty provided - use at own risk

Disclaimer

This tool is for **educational and personal use only**. The developer is not responsible for:

- Misuse of the application
- Violations of website terms of service
- Job application outcomes
- Data accuracy or completeness
- Any damages arising from use

Ethical Usage Guidelines

- ⚠️ Always provide accurate information
- ⚠️ Don't spam companies with multiple applications
- ⚠️ Respect rate limits on job portals
- ⚠️ Use for genuine job search only
- ⚠️ Review applications before submission

Thank You!

Advanced AI Job Application Bot v2.0

📞 Contact Developer

Vignesh Yadala

📧 vignesh.yadala@gmail.com

💼 linkedin.com/in/vignesh-yadala

🔗 github.com/Vigneshyadala

📱 +91 9032478898

"Automating success, one application at a time" 🚀

© 2025 Advanced AI Job Application Bot | Version 2.0 | December 2025

Made with ❤️ and Python by Vignesh Yadala